

QGIS + PostGIS: erros silenciosos que ninguém avisa

2026-09-22

@Someone

Por que esta apostila existe

A maioria dos problemas em projetos de geoprocessamento não aparece como mensagem de erro. Aparece como um filtro que retorna zero linhas, um join que não casa, um mapa deslocado 60 metros ou uma consulta que ordena "errado". A causa quase sempre é um detalhe pequeno: um acento, um espaço no fim do texto, uma letra maiúscula, um nome de pasta longo demais.

Esta apostila lista essas armadilhas em quatro lugares — o shapefile, os nomes no PostGIS, os dados dentro do PostGIS e o Windows — e termina com um exercício onde todas elas aparecem de uma vez.

Regra geral que resolve 80% dos casos: **nomes de tabela, coluna, arquivo e pasta em minúsculas, sem acento, sem espaço, usando _** (snake_case), e caminhos curtos como `C:\gis\projeto\`.

Shapefile: o que o formato perde sem avisar

O shapefile é de 1998 e guarda os atributos num arquivo DBF. Cada limite do DBF vira uma perda silenciosa na exportação.

Limite	O que acontece	Como perceber
Nome de campo: 10 caracteres	<code>nome_completo</code> vira <code>nome_compl</code> ; dois campos com o mesmo início viram <code>nome_com_1</code> , <code>nome_com_2</code>	Join por nome de campo não casa; o QGIS avisa uma vez, no export
Texto: 254 caracteres	Endereços e observações longas são cortados	<code>length()</code> no PostGIS depois do import
Sem NULL	Texto vazio vira <code>' '</code> ; número vazio vira NULL no OGR/QGIS, mas 0 no Excel/LibreOffice	<code>WHERE campo IS NULL</code> não acha nada
Sem booleano, sem data+hora	<code>true/false</code> vira <code>1/0</code> ou texto; hora é descartada	Campo <code>data_hora</code> chega só com a data

Limite	O que acontece	Como perceber
Sem inteiro 64 bits (leitores antigos)	Códigos IBGE grandes viram <code>double</code> e perdem precisão	Valor termina em <code>...000</code> ou <code>.0</code>
Codificação no <code>.cpg</code>	Sem o <code>.cpg</code> , o QGIS adivinha (System ou Latin-1)	<code>SÃ£o JosÃ©</code> , <code>S0o Jos0</code> ; filtro por nome acentuado retorna zero
Não distingue Polygon de MultiPolygon	Import numa coluna <code>MultiPolygon</code> falha; numa coluna <code>geometry</code> genérica entram tipos misturados	Marcar <i>Promover para multipartes</i> no import
<code>.prj</code> ausente ou fora do padrão	O QGIS assume um CRS ou pergunta; clicar OK sem ler abre em WGS84	Áreas e distâncias erradas; ver seção SRID
2 GB por arquivo	Export grande termina com arquivo truncado	Contagem de feições menor que a origem
3+ arquivos obrigatórios	Enviar só o <code>.shp</code> = camada sem atributos ou sem abrir	<code>.shp</code> + <code>.shx</code> + <code>.dbf</code> (+ <code>.prj</code> + <code>.cpg</code>) sempre juntos

O mais traiçoeiro é o encoding: a tabela de atributos às vezes **mostra** o acento certo e mesmo assim o filtro `"nome" = 'São José'` retorna zero linhas, porque o texto na memória é outro. Teste rápido: `length("nome")` — `São José` tem 8 caracteres; a versão corrompida tem 10 ou mais.

Alternativa moderna: **GeoPackage** (`.gpkg`) não tem nenhum desses limites, é um arquivo só, e o QGIS o usa como padrão desde a versão 3.

PostGIS: maiúsculas, aspas, espaços e acentos nos nomes

O PostgreSQL converte para minúsculas todo nome escrito **sem aspas duplas**. Com aspas duplas, ele preserva exatamente o que foi digitado — caixa, espaço e acento — e a partir daí todo SQL, view, trigger, model do Laravel e camada do GeoServer precisa das aspas para sempre.

```

CREATE TABLE Bairros (Nome_Bairro text);           -- cria bairros / nome_bairro
CREATE TABLE "Bairros" ("Nome Bairro" text);      -- cria Bairros / Nome Bairro
(outra tabela!)

SELECT * FROM bairros;                             -- lê a primeira
SELECT * FROM Bairros;                             -- também a primeira (sem aspas =
minúsculo)
SELECT * FROM "Bairros";                           -- lê a segunda
SELECT nome_bairro FROM "Bairros";                 -- ERRO: column "nome_bairro" does not
exist
SELECT "Nome Bairro" FROM "Bairros";               -- funciona

```

Armadilha	Comportamento	Regra
Aspas simples × duplas	'nome' é texto; "nome" é coluna. WHERE "nome" = "Toledo" dá <i>column "Toledo" does not exist</i>	Texto sempre entre aspas simples
Opção do QGIS Converter nomes de campo para minúsculas	Desmarcada, NOME_BAIRRO e Área m² viram colunas entre aspas	Deixar marcada
Limite de 63 bytes por nome	O PostgreSQL trunca e só emite NOTICE , não erro. Acento em UTF-8 ocupa 2 bytes, então cabe menos	Nomes curtos, sem acento
Duas colunas longas com o mesmo início	Truncam para o mesmo nome → <i>column already exists</i>	Idem
Palavras reservadas	order, desc, user, group, table, select, end, type como nome de campo quebram qualquer SQL sem aspas	Renomear no import: ordem, descricao, usuario
Nome começando com número	2023_pop só funciona como "2023_pop"	pop_2023
Conflito com a chave que o QGIS cria	O import cria id (ou fid) como PK; se o shape já tinha ID, sobra id e id_1, e ninguém sabe qual é o original	Renomear o campo original antes (cod_lote)

Armadilha	Comportamento	Regra
Schema errado / search_path	Importou em <code>public</code> , a view aponta para <code>geo.bairros</code> : <i>relation does not exist</i> com a tabela existindo	Sempre qualificar: <code>geo.bairros</code>

Verificação rápida no psql: `\d "Bairros"` mostra o nome exato de cada coluna, com aspas onde forem necessárias.

PostGIS: armadilhas dentro dos dados

Mesmo com os nomes certos, o conteúdo das colunas engana. Nenhum destes casos gera erro: só resultado diferente do esperado.

Armadilha	Exemplo	Solução
Comparação diferencia maiúsculas	<code>'TOLEDO' <> 'Toledo'</code>	<code>ILIKE, lower(nome) = 'toledo'</code>
Acento	<code>'Sao Jose' <> 'São José'</code>	Extensão <code>unaccent</code> : <code>unaccent(lower(nome)) = 'sao jose'</code>
Espaço no fim	<code>'Toledo ' <> 'Toledo'</code> — comum em dados digitados no Excel	<code>trim(nome)</code> ; ver com <code>length(nome)</code>
Vazio × NULL	Do DBF vem <code>''</code> ; <code>WHERE nome IS NULL</code> não acha	<code>WHERE nome IS NULL OR nome = ''</code> ou <code>nullif(nome, '')</code>
Número guardado como texto	<code>'100' < '20' < '9'</code> no <code>ORDER BY</code> ; soma dá erro	Converter tipo da coluna
Vírgula decimal	<code>'12,5'::numeric</code> falha	<code>replace(campo, ',', '.')</code> <code>::numeric</code>
Ordenação com acento depende da collation do banco	Em collation <code>C</code> , <code>Água</code> ordena depois de <code>Zumbi</code> ; em <code>pt_BR.UTF-8</code> ordena entre <code>A</code> e <code>B</code>	Criar o banco com <code>pt_BR.UTF-8</code> ou usar <code>COLLATE</code> na consulta
Encoding no import	<code>shp2pgsql</code> sem <code>-W latin1</code> , ou codificação errada na camada do QGIS	Mojibake silencioso, ou <code>invalid byte sequence for encoding "UTF8"</code> (esse pelo menos avisa)

```
-- Diagnóstico rápido de uma coluna de texto
SELECT nome, length(nome), nome <> trim(nome) AS tem_espaco, nome = '' AS
vazio
FROM bairros
WHERE nome <> trim(nome) OR nome = '' OR nome IS NULL;

-- Busca tolerante
CREATE EXTENSION IF NOT EXISTS unaccent;
SELECT * FROM bairros WHERE unaccent(lower(trim(nome))) = 'sao jose';

-- Collation do banco
SELECT datname, datcollate FROM pg_database WHERE datname =
current_database();
```

PostGIS + QGIS: SRID, geometria, chave primária e views

Aqui o QGIS colabora com o erro: a reprojeção em tempo real e a leitura tolerante escondem problemas que só aparecem numa medição ou num cruzamento com outro dado.

SRID assumido errado. Um shape sem `.prj` importado com *Assumir CRS = SIRGAS 2000 / UTM 22S (31982)* quando o dado era SAD69 (29192) fica deslocado cerca de 60 m — e o QGIS desenha tudo “no lugar”. SIRGAS 2000 e WGS84 diferem menos de 1 m, então ninguém percebe. Sempre confirmar a origem do dado antes de assumir.

```
SELECT DISTINCT ST_SRID(geom) FROM lotes;           -- deve retornar um
único valor, nunca 0
SELECT Find_SRID('public', 'lotes', 'geom');       -- o que a coluna
declara
```

Geometria inválida. Polígono com auto-interseção (“gravata borboleta”) ou anel não fechado. `ST_Intersects`, `ST_Within` e `ST_Contains` **retornam resultado errado sem avisar**; só `ST_Union` e `ST_Intersection` estouram com *TopologyException*.

```
SELECT count(*) FROM lotes WHERE NOT ST_IsValid(geom);
SELECT id, ST_IsValidReason(geom) FROM lotes WHERE NOT ST_IsValid(geom);
UPDATE lotes SET geom = ST_MakeValid(geom) WHERE NOT ST_IsValid(geom);
```

Tabela sem chave primária inteira. O QGIS abre a camada como somente leitura, ou usa o `ctid` interno como chave — e a edição “funciona” até um `VACUUM` renumerar os `ctids`. Em views, o QGIS pede uma coluna única; escolher uma que não é única faz feições sumirem ou aparecerem duplicadas no mapa.

Metadados estimados. A conexão do QGIS tem a opção *Usar metadados estimados da tabela*: extensão e contagem vêm das estatísticas do PostgreSQL. Tabela recém-carregada sem `ANALYZE` → *Zoom para a camada* vai para o lugar errado ou a camada parece vazia.

```
ANALYZE lotes; -- depois de todo import grande
```

View sem tipo de geometria explícito. O QGIS varre a view inteira para descobrir o tipo (lento em tabelas grandes) ou trata como "desconhecido" e não estiliza. Sempre declarar o tipo e o SRID na view:

```
CREATE VIEW v_lotes AS
SELECT id, inscricao, geom::geometry(MultiPolygon, 31982) AS geom
FROM lotes;
```

Tipos misturados numa coluna `geometry` genérica. Polygon e MultiPolygon na mesma coluna: o QGIS mostra só um dos tipos e não avisa. Ver com `SELECT DISTINCT GeometryType(geom) FROM tabela`.

Windows: pastas, nomes e caminhos

A regra prática: projeto em `C:\gis\projeto\`, nunca em `C:\Users\João Silva\OneDrive - Prefeitura\Documentos\Secretaria\Geoprocessamento\2026\...`.

Armadilha	O que acontece	Regra
Caminho com mais de 260 caracteres (MAX_PATH)	Vale na prática para a maioria dos programas, mesmo com <i>LongPathsEnabled</i> no registro. Sintoma: "não foi possível abrir o arquivo", ou shapefile salvo pela metade (o <code>.shp</code> grava, um auxiliar não), ou pasta que o Explorer não consegue apagar	Caminhos curtos, poucos níveis

Armadilha	O que acontece	Regra
Acento ou espaço no caminho (C:\Users\João Silva\)	GDAL e QGIS núcleo funcionam; GRASS, SAGA e vários plugins Python não. Sintoma: algoritmo do Processing termina com saída vazia ou "arquivo não encontrado" apontando para um caminho cortado no espaço	Pasta de trabalho sem acento e sem espaço
OneDrive / Google Drive sincronizando a pasta	Arquivo .lock , .dbf gravado durante a sincronização, projeto .qgz corrompido, "Documentos" redirecionado para o OneDrive sem o usuário saber	Trabalhar fora da pasta sincronizada; copiar para lá só o resultado
Windows ignora maiúsculas, Linux não	Bairros.shp + bairros.dbf funciona no Windows e quebra no servidor (GeoServer, QGIS Server, API)	Tudo em minúsculas
ZIP com acento no nome de arquivo	Zipado no Linux (UTF-8), aberto no Explorer (CP437): ãrea.shp ; o QGIS não acha o .dbf porque o nome não bate mais	Nomes ASCII dentro do ZIP
Caminho absoluto no projeto QGIS	Pasta movida → camadas "indisponíveis" (visível). Invisível: uma cópia antiga do shape ainda existe no caminho absoluto e o projeto abre com o dado velho	Salvar o projeto com caminhos relativos (Propriedades do projeto → Geral)
Arquivos temporários do Processing em %TEMP%	Saídas temporárias somem ao fechar o QGIS; um %TEMP% com caminho longo agrava o problema do MAX_PATH	Sempre salvar a saída num arquivo nomeado

Checklist dos erros silenciosos

Erros que não geram mensagem — só resultado errado. Quando algo "não bate", percorrer esta lista antes de desconfiar do dado.

#	Sintoma	Causa provável	Verificação
1	Filtro ou join retorna zero linhas	Acento, maiúscula, espaço no fim, '' em vez de NULL	<code>length()</code> , <code>trim()</code> , <code>unaccent()</code> , <code>lower()</code>
2	Campo "sumiu" ou tem outro nome	Truncado a 10 caracteres (shape) ou 63 bytes (PostgreSQL)	<code>\d tabela</code> ; ler o aviso do export
3	Texto termina no meio	Limite de 254 caracteres do DBF	<code>max(length(campo))</code> na origem × destino
4	Mapa deslocado dezenas de metros	CRS assumido errado no import, reprojetado em tela	<code>ST_SRID(geom)</code> ; comparar com um dado de referência
5	Cruzamento espacial deixa feições de fora	Geometria inválida em predicado (<code>ST_Intersects</code>)	<code>ST_IsValid(geom)</code>
6	<code>ORDER BY</code> fora de ordem	Collation <code>C</code> , ou número guardado como texto	<code>datcollate</code> ; tipo da coluna
7	Zoom para a camada vai para o lugar errado	Metadados estimados sem <code>ANALYZE</code>	<code>ANALYZE tabela</code>
8	Processing gera saída vazia	Acento ou espaço no caminho (GRASS/SAGA)	Mover para <code>C:\gis\</code> e repetir
9	Edição feita, depois perdida	Tabela sem chave primária inteira (QGIS usa <code>ctid</code>)	<code>\d tabela</code> — precisa de PK
10	Projeto abre com dado antigo	Caminho absoluto apontando para cópia velha	Propriedades da camada → Fonte
11	Só parte das feições aparece	Polygon e MultiPolygon misturados na coluna	<code>SELECT DISTINCT GeometryType(geom)</code>
12	Camada abre, mas sem atributos	Faltou o <code>.dbf</code> , ou <code>.shp</code> e <code>.dbf</code> com caixa diferente no Linux	Conferir os arquivos auxiliares

Exercício prático: o shapefile sujo

O kit da aula traz `bairros_sujo.shp`, um shapefile com todas as armadilhas acima ao mesmo tempo, e `bairros_fonte.geojson`, a versão limpa da origem. O objetivo é importar no PostGIS e responder três perguntas — cada uma esbarra em pelo menos uma armadilha.

O que o arquivo tem de errado (não contar para a turma antes):

- Sem `.cpg` (o texto está em Latin-1) e sem `.prj` (o dado está em SIRGAS 2000 / UTM 22S, EPSG 31982)
- Campo chamado `ORDER` (palavra reservada) e campo `NOME` em maiúsculas
- Um nome com espaço no fim, um nome vazio (`' '`), nomes com acento
- `AREA_HA` e `POP` guardados como texto, um deles com vírgula decimal
- Um polígono inválido (auto-interseção) e uma feição multipartes
- No GeoJSON de origem: nome de campo com mais de 10 caracteres e uma descrição com mais de 254 caracteres, para o exercício de exportação

Passos:

1. Descompactar o kit em `C:\Users\<seu usuário>\Área de Trabalho\Aula QGIS\` (com acento e espaço, de propósito).
2. Abrir `bairros_sujo.shp` no QGIS. Observar: que CRS o QGIS assumiu? Como aparecem os nomes na tabela de atributos?
3. Corrigir a codificação nas propriedades da camada (Fonte → Codificação → ISO-8859-1) e definir o CRS como EPSG 31982.
4. Importar para o PostGIS pelo DB Manager. Testar com e sem *Converter nomes de campo para minúsculas* e com e sem *Promover para multipartes*.
5. Rodar `demo.sql` bloco a bloco e comparar com o gabarito nos comentários.
6. Responder:
 - Quantos bairros se chamam "São José"? (o esperado é 2; a busca ingênua acha 0 ou 1)
 - Qual bairro tem a maior área em hectares? (ordenar como texto dá a resposta errada)
 - Quais bairros fazem fronteira com "Centro"? (o polígono inválido fica de fora até o `ST_MakeValid`)
7. Exportar `bairros_fonte.geojson` para shapefile e conferir os nomes de campo e o tamanho da descrição.
8. Rodar um algoritmo do GRASS (por exemplo *v.buffer*) com a camada na pasta com acento; depois mover para `C:\gis\` e repetir.

O ponto da aula não é decorar a lista: é criar o hábito de perguntar "o que esse resultado silencioso pode estar escondendo?" antes de aceitar zero linhas, um mapa bonito ou uma ordenação estranha.